

Übungsblatt 7

Im Moodle finden Sie einen Interpreter, der eine einfache Programmiersprache namens MJava, die den `split`-Befehl unterstützt, simulieren kann. Der Interpreter ist in Java geschrieben, Sie können ihn also nur benutzen, wenn Sie Java (Version 1.8 oder höher) auf Ihrem Rechner installiert haben. Der Interpreter kann von Shell/Kommandozeile wie folgt gestartet werden:

```
java -jar mjava.jar <Programm>
```

Dabei ist `<Programm>` der Dateiname eines MJava-Programms.

Die Sprache MJava sollte keine großen Hürden für Menschen darstellen, die bereits C oder C++ oder Java können, da es sich um eine stark abgespeckte Variante dieser Sprachen handelt. Es handelt sich nicht um eine Objektorientierte Sprache, Klassen können also nicht definiert werden. Das komplexeste zulässige Konstrukt sind Funktionen, welche aber wie gewohnt definiert werden dürfen. (Deklarationen von Funktionen ohne zugehörige Definition ist nicht erlaubt. Es können keine externen Programmdateien (via `include` oder `import`) eingebunden werden.

In der Sprache ist es auch nicht möglich komplexe Datentypen zu entwerfen. Folgende Typen sind aber zulässig:

- `int`
- `long`
- `float`
- `double`
- `String`
- `boolean`

Diese verhalten sich wie gewohnt. Strings können, wie in Java, unter Verwendung von `+` konkateniert werden. Strings können mit dem Operator `+` auch mit Ausdrücken eines anderen Typs konkateniert werden. Z.B. ist das Ergebnis des Ausdrucks `"bla" + 7` gleich dem String `"bla7"`. Zur Ausgabe auf Konsole dient eine interne Funktion `println` (z.B.: `println("Hello World!")`).

Arrays über obige Datentypen sind erlaubt. Ein neues Array kann wie in Java wie folgt deklariert und initialisiert werden:

- `int[] a = new int[10];`
Erzeugt ein `int`-Array mit 10 Elementen, die alle mit 0 initialisiert sind.
- `int[] a = new int[]{5, 6, 7};`
Erzeugt ein `int`-Array mit 3 Elementen, die mit den Werten 5, 6 und 7 initialisiert sind.

In MJava kann die Länge eines Arrays über die eingebaute Funktion `length` bestimmt werden. Ist also z.B. `a` wie oben (2. Beispiel) initialisiert, so liefert der Ausdruck `length(a)` den Wert 3 zurück.

In MJava gibt es die `split`-Anweisung, die die Berechnung auf zwei Maschinen aufteilt. Die Syntax des `split`-Befehls ist wie folgt:

```
split <Wertzuweisung 1> and <Wertzuweisung 2>
```

Dabei sind `<Wertzuweisung 1>` und `<Wertzuweisung 2>` Wertzuweisungen der Form

```
<Variable> = <Wertausdruck> ;
```

Beispiel: Der folgende Code ist eine gültige `split`-Anweisung:

```
split x = 5; and x = 7;
```

Sie bewirkt, dass nach ihrer Ausführung zwei parallele Rechenwege existieren, wobei im ersten dieser Rechenwege die Variable `x` den Wert 5 annimmt und im zweiten Rechenweg den Wert 7.

Die Wirkung des `split`-Befehls besteht darin, dass vor Ausführung von `<Wertzuweisung 1>` und `<Wertzuweisung 2>` zwei identische Kopien der ausführenden Maschine gemacht werden (also inklusive Speicherinhalt und aktueller Zustand des Programms) die danach parallel weiterarbeiten wobei die eine mit `<Wertzuweisung 1>` und die andere mit `<Wertzuweisung 2>`.

Mit Hilfe des `split`-Befehls können also mehrere parallele Berechnungspfade erzeugt werden. Ein MJava-Programm hält akzeptierend, falls die `main`-Methode den Rückgabotyp `boolean` besitzt und es mindestens einen parallelen Rechenweg gibt, der dazu führt, dass die Rückgabe der `main`-Methode den Wert `true` besitzt. In allen anderen Fällen hält ein MJava-Programm ablehnend.

Aufgaben:

1. Im Moodle finden Sie auch eine Reihe von `.mjava` Programmen. Sehen Sie sich das Programm `easy.mjava` und überlegen Sie sich was es tun und ausgeben wird. Starten Sie über Kommandozeile dieses Programm indem Sie

```
java -jar mjava.jar easy.mjava
```

eingeben. Verstehen Sie die Ausgabe und wie sie mit dem Programm zusammenhängt.

2. Betrachten Sie nun das Programm `counting.mjava`. Überlegen Sie, was es tut und was seine Ausgabe sein wird! Lassen Sie das Programm laufen und beantworten sie folgende Fragen:

- (a) Warum hält das Programm ablehnend?
- (b) Was ist der größte Wert, den sie statt 1023 einsetzen können, so dass das Programm akzeptierend hält. Warum können Sie keinen größeren/ kleineren Wert wählen?
- (c) Wie viele unterschiedliche Werte werden per `println` ausgegeben?
- (d) Erklären Sie, warum die Laufzeit des Programms nur 67 Schritte ist, obwohl über 1000 Ausgaben gemacht werden!
- (e) Wie lange müsste die Schleife mindestens laufen, damit über 64000 Ausgaben gemacht werden?

3. Es sei das Problem PARTITION wie folgt definiert:

$$\text{PARTITION} \stackrel{\text{def}}{=} \{(c_1, \dots, c_n) : \text{es gibt ein } I \subseteq \{1, \dots, n\}, \text{ mit } \sum_{i \in I} c_i = \sum_{i \notin I} c_i\}$$

Beispiele:

- $(5, 3, 6, 7, 2, 8, 12, 13) \in \text{PARTITION}$, weil für $I = \{2, 7, 8\}$ gilt: $\sum_{i \in I} c_i = 28 = \sum_{i \notin I} c_i$ wobei $c_1 = 5, c_2 = 3, c_3 = 6, c_4 = 7, c_5 = 2, c_6 = 8, c_7 = 12, c_8 = 13$.
- $(11, 12, 15, 4, 19, 8, 16, 3) \notin \text{PARTITION}$, weil es kein $I \subseteq \{1, \dots, 8\}$ gibt, so dass $\sum_{i \in I} c_i = \sum_{i \notin I} c_i$.

Schreiben Sie ein nichtdeterministisches MJava-Programm, das das PARTITION-Problem in Polynomialzeit löst! Schreiben Sie dazu am besten eine Methode mit dem Kopf

```
boolean inPARTITION(int n, int[] a)
```

die `true` zurückgibt, falls sie eine Lösung für das gegebene PARTITION-Problem gefunden hat. Benutzen Sie den Nichtdeterminismus und bedenken Sie, dass sie mitunter verschiedene parallele Rechenwege am Laufen haben (falls Sie den `split`-Befehl einsetzen) und dass es genügt auf einem der parallelen Rechenwege eine Lösung zu finden.

Testen Sie Ihr Programm an den oben aufgeführten Beispielen!

4. (a) Zeigen Sie: $\text{PARTITION} \leq \text{SOS}$
(b) Zeigen Sie: $\text{SOS} \leq \text{PARTITION}$